

Chương 3

Phân loại tài liệu được mã hóa qua kỹ thuật Bag-of-Words (BoW) sử dụng Multinomial Naive Bayes (MNB)

§1 Bài toán & trực giác

Mục tiêu: Cho một văn bản d và một tập nhãn chủ đề $C = \{c_1, \dots, c_K\}$ (vd. 1: *thể thao*; 2: *công nghệ*; ...).

Dựa vào "nội dung văn bản" được biểu thị qua các từ có trong đó, hãy dự đoán nhãn $\hat{c} = \arg \max_{c \in C} p(c | d)$.

Trực giác của MNB. MNB giả định:

- Mỗi lớp c có một phân bố từ vừng riêng $\theta_c = (\theta_{1c}, \dots, \theta_{Vc})$ trên V từ trong vốn từ (vocabulary).
- Một văn bản là "túi từ" (BoW): chỉ đếm số lần mỗi từ xuất hiện, không quan tâm thứ tự.
- Các từ xuất hiện độc lập có điều kiện: chỉ xét văn bản thuộc một lớp cho trước (giả định "naive").

Điều này dẫn đến mô hình sinh (generative) mà trong đó *đếm từ* trong văn bản được mô tả tự nhiên bởi *phân phối đa thức (multinomial)*; do đó phù hợp trực tiếp với BoW.

§2 Kỹ thuật Bag-of-Words (BoW) để trích xuất vector đặc trưng

2.1 Mục tiêu, ký hiệu và ánh xạ đặc trưng

Kí hiệu vốn từ $\mathcal{V} = \{1, \dots, V\}$. Với mỗi tài liệu d , BoW ánh xạ $\varphi : d \mapsto x \in \mathbb{R}^V$ sao cho thành phần x_j tóm tắt “mức hiện diện” của từ j trong d (đếm, nhị phân, tf, tf-idf...). Để dùng với MNB, khuyến nghị $x_j \in \mathbb{N}$ (đếm nguyên) hoặc tf tỉ lệ với đếm.

2.2 Quy trình tổng quát

- i. **Chuẩn hoá văn bản:** mã Unicode (NFC/NFKC), chuyển chữ thường (nếu phù hợp), xử lý dấu câu, số, URL, email, emoji...
- ii. **Tách từ (tokenize):** theo từ, cụm từ, hoặc ký tự (char n-gram).
- iii. **Lọc token:** loại stopwords, token quá ngắn/dài, ký hiệu thuần tuý.
- iv. **Sinh n-gram:** word n-grams (1–3), hoặc char n-grams (3–5, hoặc char_wb).
- v. **Xây vốn từ:** chọn các n-gram thoả `min_df`, `max_df`, `max_features`.
- vi. **Mã hoá:** ánh xạ token $\rightarrow$ chỉ số và đếm xuất hiện để tạo ma trận thưa $X \in \mathbb{R}^{N \times V}$.
- vii. **Cân nhắc trọng số:** chọn sơ đồ (đếm, nhị phân, tf, log-tf, tf-idf), và **chuẩn hoá** (L1/L2) nếu dùng các mô hình tuyến tính khác.

2.3 Tokenization cho tiếng Việt

Tiếng Việt cần **tách từ** (word segmentation) vì khoảng trắng phân tách *tiếng* chứ không luôn là *từ*. Công cụ phổ biến: *VnCoreNLP*, *underthesea*, *pyvi*. Nên:

- Ghép từ đa tiếng bằng gạch dưới: `tr_í_tuệ_nhân_tạo`, `bóng_đá`.
- Chuẩn hoá Unicode (NFC), giữ dấu (dấu thanh mang nghĩa).
- Cân nhắc danh sách stopwords tiếng Việt, nhưng kiểm tra thủ công để tránh xoá từ quan trọng theo ngữ cảnh.

2.4 Xây vốn từ & cắt tỉa (pruning)

- `min_df`: loại từ quá hiếm (giảm nhiễu, tăng tổng quát hoá).
- `max_df`: loại từ quá phổ biến (gần như dùng từ).
- `max_features`: giới hạn kích thước V (bộ nhớ/tốc độ).
- **Chống rò rỉ**: xây vocab **chỉ** từ tập train; áp dụng lại nguyên vẹn cho dev/test.

2.5 n-grams: word vs. character

- **Word n-grams** (1–3): bắt ngữ cảnh ngắn (`học_máy`, `mạng_nơ_ron`).
- **Character n-grams**: bền vững trước lỗi chính tả, biến thể hình thái; hữu ích cho tiếng Việt và dữ liệu bản. `char_wb` (n-gram trong ranh giới từ) giúp tránh n-gram bắc cầu khoảng trắng.

2.6 Sơ đồ trọng số (weighting schemes)

Giả sử $tf_j(d)$ = số lần từ j xuất hiện trong d , $n_d = \sum_j tf_j(d)$, df_j = số tài liệu chứa j , N = số tài liệu.

- **Đếm (Counts)**: $x_j = tf_j(d)$. Phù hợp nhất cho MNB.
- **Nhị phân (Binary)**: $x_j = \mathbf{1}\{tf_j(d) > 0\}$. Gần với Bernoulli NB.
- **TF chuẩn hoá**: $x_j = tf_j(d)/n_d$ (giảm ảnh hưởng độ dài).
- **Log-tf (sublinear)**: $x_j = 1 + \log tf_j(d)$ nếu $tf_j(d) > 0$, ngược lại 0.
- **TF-IDF (smooth)**: $x_j = tf_j(d) \cdot \left(\log \frac{N+1}{df_j+1} + 1\right)$. Thực nghiệm tốt, nhưng lệch mô hình MNB.
- **Chuẩn hoá vector**: ℓ_1 hoặc ℓ_2 (hữu ích cho SVM/LR). Với MNB, ưu tiên *không* chuẩn hoá sau đếm.

2.7 Hashing trick (tuỳ chọn, dữ liệu lớn)

Không cần lưu vocab tường minh: ánh xạ mỗi token về chỉ số $h(\text{token}) \bmod D$. Ưu: bộ nhớ cố định, hỗ trợ online. Nhược: *xung đột* (collisions), khó diễn giải. Dùng `HashingVectorizer`.

2.8 Ma trận thưa & độ phức tạp

Ma trận X cực thưa; lưu CSR/CSC. Vector hoá có độ phức tạp gần tuyến tính theo tổng số token. Với MNB, suy diễn là tích vô hướng trên các chỉ số j với $x_j > 0 \Rightarrow$ rất nhanh.

2.9 OOV (từ ngoài vốn từ)

Token không có trong vocab được bỏ qua (xem như $x_j = 0$). Tùy chọn tạo bucket <UNK> khi xây vocab để gom từ hiếm mới.

2.10 Ví dụ cấu hình với scikit-learn (tiếng Việt)

```
from underthesea import word_tokenize
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

def vn_tokenizer(text):
    return word_tokenize(text, format="text").split()

vectorizer = CountVectorizer(
    tokenizer=vn_tokenizer,          # tách từ tiếng Việt
    lowercase=True,                 # chuyển hết sang chữ viết thường
    ngram_range=(1,2),              # uni+bi-gram theo từ
    min_df=2, max_df=0.9,           # cắt tỉa các từ ít xuất hiện
    max_features=80000              # giới hạn vocab
    # analyzer="word" (mặc định); với char n-gram: analyzer="char_wb",
    ngram_range=(3,5)
)

clf = make_pipeline(vectorizer, MultinomialNB(alpha=0.5))
clf.fit(X_train_texts, y_train)
y_pred = clf.predict(X_valid_texts)
```

2.11 Ví dụ đếm đơn giản

Giả sử $D = \{\text{"bóng_đá việt_nam thắng"}, \text{"trí_tuệ_nhân_tạo việt_nam"}\}$. Vocab sau khi tách: $\{\text{bóng_đá, việt_nam, thắng, trí_tuệ_nhân_tạo}\}$. Ma trận đếm X :

Tài liệu	bóng_đá	việt_nam	thắng	trí_tuệ_nhân_tạo
d_1	1	1	1	0
d_2	0	1	0	1

Đây là đúng kiểu đầu vào mong đợi của MNB (đếm nguyên, không âm).

2.12 Thực hành tốt & kiểm soát chất lượng

- Giữ một tập *dev* riêng để điều chỉnh `min_df`, `max_df`, `ngram_range`, α của MNB.
- Kiểm tra top từ theo lớp dựa trên $\log \theta_{jc}$ để phát hiện token rác.
- Gộp các dạng viết tương đương (HCM, TP.HCM, TPHCM) bằng tiền xử lý.
- Tránh rò rỉ: không vector hoá test bằng vocab xây từ toàn bộ dữ liệu.

§3 Vì sao MNB phù hợp với bài toán phân loại văn bản đã mã hóa bằng BoW?

Ký hiệu và mô hình sinh

Giả sử từ tập tài liệu, chúng ta sử dụng kỹ thuật BoW và đã tổng hợp được:

- Vốn từ vựng (bảng từ điển) $\mathcal{V} = \{1, \dots, V\}$.
- Tài liệu d gồm n token $w_1, \dots, w_n, w_i \in \mathcal{V}$.
- Vector đếm BoW $x = (x_1, \dots, x_V) \in \mathbb{N}^V$ với $x_j = \#\{w_i = j\}$, $\sum_j x_j = n$.
- Các phân lớp $c \in C = \{c_1, \dots, c_K\}$, và từ đó tính được xác suất tiên nghiệm $\pi_c = p(c)$, $\sum_c \pi_c = 1$.
- Tham số theo lớp $\theta_c = (\theta_{1c}, \dots, \theta_{Vc})$, $\theta_{jc} \geq 0$, $\sum_j \theta_{jc} = 1$.

Mô hình sinh:

- (1) chọn $c \sim \text{Categorical}(\pi)$;
- (2) chọn độ dài n (độc lập với θ);
- (3) với $i = 1 \dots n$, rút ra $w_i \sim \text{Categorical}(\theta_c)$, độc lập có điều kiện theo từng lớp.

Từ chuỗi token đến vector đếm

Xác suất một chuỗi $w_{1:n}$ cho lớp c : $p(w_{1:n} | c) = \prod_{i=1}^n \theta_{w_i c}$. Gom theo đếm x_j : $\prod_{i=1}^n \theta_{w_i c} = \prod_{j=1}^V \theta_{jc}^{x_j}$.

Vì BoW không quan tâm thứ tự, số hoán vị cho cùng x là $\frac{n!}{\prod_j x_j!}$. Tổng xác suất mọi chuỗi có cùng x :

$$p(x | c) = \frac{n!}{\prod_{j=1}^V x_j!} \prod_{j=1}^V \theta_{jc}^{x_j},$$

chính là phân phối đa thức $x \sim \text{Multinomial}(n, \theta_c)$.

Vì sao bỏ được hệ số tổ hợp khi phân loại

Ta chọn $\hat{c} = \arg \max_c p(c | x) = \arg \max_c p(x | c) \pi_c$. Lấy log:

$$\hat{c} = \arg \max_c \left[\log \pi_c + \log \frac{n!}{\prod_j x_j!} + \sum_{j=1}^V x_j \log \theta_{jc} \right].$$

Hạng $\log \frac{n!}{\prod_j x_j!}$ không phụ thuộc c , nên bị triệt khi lấy arg max. Do đó:

$$\hat{c} = \arg \max_c \left(\log \pi_c + \sum_{j=1}^V x_j \log \theta_{jc} \right).$$

Quy tắc quyết định tuyến tính theo đếm x , với trọng số $\log \theta_{jc}$ và chặn $\log \pi_c$.

Diễn giải log-odds (hai lớp)

Với hai lớp c, c' :

$$\log \frac{p(c | x)}{p(c' | x)} = \log \frac{\pi_c}{\pi_{c'}} + \sum_{j=1}^V x_j \log \frac{\theta_{jc}}{\theta_{jc'}}.$$

Mỗi từ j đóng góp một trọng số $\log(\theta_{jc}/\theta_{jc'})$; nếu $\theta_{jc} > \theta_{jc'}$ thì xuất hiện x_j lần sẽ đẩy điểm về phía c .

Vai trò độ dài tài liệu n

Nếu dùng tần suất $f_j = x_j/n$, thì $\text{score}(c) = \log \pi_c + n \sum_j f_j \log \theta_{jc}$. Hệ số n thể hiện nhiều bằng chứng hơn $\Rightarrow$ độ tin lớn hơn.

Ước lượng θ & làm tròn Dirichlet

Ước lượng tần suất thuần: $\hat{\theta}_{jc} = N_{jc}/T_c$. Tránh xác suất 0 bằng làm tròn Dirichlet (Lidstone/Laplace):

$$\hat{\theta}_{jc} = \frac{N_{jc} + \alpha}{T_c + \alpha V}, \quad \alpha > 0.$$

Diễn giải như cộng số đếm ảo α cho mỗi từ; bảo đảm $\log \theta_{jc}$ hữu hạn.

BoW đếm vs. TF-IDF

MNB đúng mô hình với đếm nguyên x_j (hoặc tf tỉ lệ với đếm). Dùng TF-IDF có thể hiệu quả thực nghiệm nhưng không còn diễn giải đa thức chuẩn, dù vẫn hoạt động như bộ phân loại tuyến tính nếu giá trị không âm.

Tóm tắt: BoW cung cấp chính xác vector đếm mà phân phối đa thức của MNB cần; tính bất biến theo thứ tự và quy tắc quyết định tuyến tính khiến BoW “khớp” với MNB cả về toán học lẫn trực giác.

§4 Chuẩn bị dữ liệu & biểu diễn BoW

- i. **Tiền xử lý:** lowercase; chuẩn hoá Unicode; tokenize; loại dấu câu/số/ký hiệu; (tùy chọn) stopwords, stemming/lemmatization.
- ii. **Xây vốn từ:** chọn top- V theo tần suất hoặc ngưỡng min_df/max_df.
- iii. **Mã hoá BoW:** văn bản $d \rightarrow x \in \mathbb{N}^V$ (đếm) hoặc tf. MNB chuẩn tắc thích *đếm*.
- iv. **Chia tập:** train/validation/test, giữ phân bố lớp (stratify).

§5 Huấn luyện MNB (ước lượng tham số)

Kí hiệu: N_{jc} là tổng số lần từ j xuất hiện trong *mọi* văn bản lớp c ; $T_c = \sum_{j=1}^V N_{jc}$; n_c là số văn bản lớp c , $n = \sum_c n_c$.

Ước lượng MAP với làm trơn Dirichlet (Lidstone/Laplace):

$$\theta_{jc} = \frac{N_{jc} + \alpha}{T_c + \alpha V}, \quad \alpha > 0.$$

Xác suất tiên nghiệm lớp:

$$\pi_c = p(c) = \frac{n_c + \beta}{n + \beta K}, \quad \beta \geq 0.$$

Tính chất: đếm qua dữ liệu $O(\#token)$; lưu trữ $O(KV)$ (thưa).

§6 Suy diễn (dự đoán)

Với văn bản mới x :

$$\text{score}(c) = \log \pi_c + \sum_{j=1}^V x_j \log \theta_{jc}.$$

Tính trên log để tránh underflow; dùng vector thưa; xử lý OOV bằng cách bỏ qua hoặc dùng <UNK> khi xây vocab.

§7 Ví dụ tính tay nhỏ gọn

Hai lớp: *Sports* (S) và *Tech* (T). Vốn từ $V = 8$: {team, win, game, score, software, code, bug, release}. Chọn $\alpha = 1$, $\pi_S = \pi_T = \frac{1}{2}$.

S (6 token): team= 2, win= 1, game= 2, score= 1. Mẫu số $6 + 8 = 14$.

$\theta_{\text{team},S} = 3/14$, $\theta_{\text{win},S} = 2/14$, $\theta_{\text{game},S} = 3/14$, $\theta_{\text{score},S} = 2/14$; còn lại $1/14$.

T (6 token): software= 1, code= 2, bug= 2, release= 1. Mẫu số $6 + 8 = 14$.

$\theta_{\text{code},T} = 3/14$, $\theta_{\text{bug},T} = 3/14$, $\theta_{\text{software},T} = 2/14$, $\theta_{\text{release},T} = 2/14$; còn lại $1/14$.

Dự đoán cho “team code win” (team= 1, code= 1, win= 1):

$$\text{Score}(S) \propto \log \frac{1}{2} + \log \frac{3}{14} + \log \frac{1}{14} + \log \frac{2}{14},$$

$$\text{Score}(T) \propto \log \frac{1}{2} + \log \frac{1}{14} + \log \frac{3}{14} + \log \frac{1}{14}.$$

Hiệu số = $\log 2 > 0 \Rightarrow$ dự đoán **Sports**.

§8 Quy trình đầy đủ (từng bước)

i. Làm sạch & tách từ.

ii. Xây vocab (min_df, max_df).

- iii. Mã hoá BoW: ma trận thưa $X \in \mathbb{N}^{N \times V}$.
- iv. Ước lượng: $N_{jc} = \sum_{i: y_i=c} X_{ij}$; $T_c = \sum_j N_{jc}$; $\theta_{jc} = (N_{jc} + \alpha)/(T_c + \alpha V)$; $\pi_c = (n_c + \beta)/(n + \beta K)$.
- v. Suy diễn: $\text{score}(c) = \log \pi_c + x^\top \log \theta_c$.
- vi. Đánh giá: accuracy/F1/macro-F1, ma trận nhầm lẫn, ROC-AUC (nhị phân).

§9 Lựa chọn & biến thể thực tế

- α (làm trơn) tinh chỉnh trên validation ($\{0.1, 0.5, 1, 2\}$).
- Thêm n-gram (bi-/tri-gram) để nắm ngữ cảnh ngắn, kiểm soát V .
- TF vs TF-IDF: MNB “đúng mô hình” với đếm/tf; TF-IDF có thể hiệu quả thực nghiệm nhưng lệch mô hình sinh.
- Bernoulli NB (đặc trưng nhị phân) cho văn bản ngắn; Complement NB tốt khi mất cân bằng lớp.
- Chọn đặc trưng: chi-square, mutual information để giảm V .

§10 Lưu ý cho tiếng Việt

Tokenize ghép từ (“bóng đá”, “trí tuệ nhân tạo”), danh sách stopwords tiếng Việt, chuẩn hoá Unicode (NFC), giữ dấu.

§11 Bẫy thường gặp & mẹo

Luôn làm trơn ($\alpha > 0$), tính trên log, xử lý OOV, tránh rò rỉ dữ liệu (vocab chỉ từ train), cân bằng lớp nếu cần.

§12 Giả mã (pseudocode)

TRAIN

build_vocab(D_train) -> V

X_train = bow_transform(D_train, V) # sparse counts

```
for c in classes:
    N_jc = sum_rows(X_train[y==c])  # vector size V
    T_c = sum(N_jc)
    theta_c = (N_jc + alpha) / (T_c + alpha*V)
    log_theta_c = log(theta_c)
pi = (count_per_class + beta) / (n + beta*K)
log_pi = log(pi)

# PREDICT for doc x (sparse)
for c in classes:
    score[c] = log_pi[c] + dot(x, log_theta_c)
return argmax(score)
```
